DeepWalk-Enhanced Link Prediction for Large-Scale Social Networks: A Hybrid Embedding + Heuristics Model on the Twitter-2010 Graph

Dhrumil Rana
Department of Computer Science and Engineering
University of Texas at Arlington
Email: dxr2109@mavs.uta.edu

***Abstract*—Link prediction is a core task in network science with applications in online social networks, recommender systems, spam detection, and information retrieval. Classical heuristics such as Common Neighbors and Jaccard Coefficient operate only on local neighborhoods and fail to capture long-range structural dependencies in large, sparse graphs such as Twitter. In contrast, random-walk-based embeddings such as DeepWalk and Node2Vec can learn latent structural representations that encode both local and global connectivity patterns.**

In this research, we develop a hybrid link prediction system combining DeepWalk embeddings with classical graph heuristics and a Random Forest classifier. Using a one-million–edge subset of the Twitter-2010 graph, we generate truncated random walks and train a 256-dimensional skip-gram model with negative sampling to derive embeddings for 400,000 nodes. These embeddings, combined with graph heuristics (Common Neighbors, Jaccard, Adamic–Adar, Resource Allocation), form a 1028-dimensional feature vector used for supervised edge classification.

Our proposed model achieves an AUC of 0.8872, significantly outperforming the Common Neighbors baseline (AUC = 0.50). We further develop an interactive link prediction engine and a top-N recommendation module that emulate "Who to Follow" functionality in real-world systems. Our findings confirm that embedding-driven approaches produce strong performance for link prediction in large-scale social networks and demonstrate that combining embeddings with heuristics yields robust, interpretable models.

***Index Terms*—DeepWalk, Link Prediction, Graph Mining, Twitter, Random Walks, Network Embeddings, Machine Learning, Recommendation Systems.**

I. Introduction

Online social networks such as Twitter, Instagram, and LinkedIn are dynamic environments where users continuously form new relations (e.g., follow, friend, connect). Predicting which new links are likely to form in the future—the *link prediction* problem—is fundamental to:

- **Recommendation systems**: E.g., "Who to Follow" or "People You May Know" suggestions.
- **Network completion**: Filling in missing or noisy edges in partially observed graphs.
- **Fraud and spam detection**: Identifying abnormal connection patterns.
- **Community evolution**: Understanding how groups form and evolve over time.

However, large-scale social graphs such as Twitter bring several challenges:

1) **Scale**: Tens of millions of nodes and billions of edges.
2) **Sparsity**: Most nodes have low degree; many potential links are never realized.
3) **Heterogeneity**: Users differ widely in behavior, interests, and connectivity.
4) **Complex structure**: Communities and structural roles span multiple hops.

Traditional link prediction relies on local heuristics such as Common Neighbors (CN), Jaccard Coefficient, Adamic–Adar (AA), and Resource Allocation (RA). While these are simple and interpretable, they ignore global structural information and degrade in performance on large, sparse networks.

Representation learning methods such as DeepWalk [1] and Node2Vec [2] address this by learning dense vector embeddings for nodes based on random walks. These embeddings can encode multi-hop structural context, community membership, and structural roles.

A. Problem Statement

Let $G = (V, E)$ be a graph where V is the set of nodes (users) and E is the set of observed edges (follows). The link prediction task is to estimate, for each pair $(u, v) \in V \times V$ such that $(u, v) \notin E$, the probability that an edge will form in the future:

$$s(u, v) \approx \Pr\big((u, v) \text{ will exist in future} \mid G\big).$$

In this work we focus on *undirected* link prediction (existence of any connection between u and v), and construct a binary classifier that discriminates between existing and non-existing edges based on structural features.

B. Research Questions

We address the following questions:

- **Q1:** Can DeepWalk embeddings trained on a large, real-world Twitter graph significantly outperform classical link prediction heuristics?
- **Q2:** Does combining embeddings with heuristics yield better performance than embeddings alone?
- **Q3:** Can the learned model be used to build practical tools such as "Who to Follow"-style recommendations?

C. Contributions

Our main contributions are:

- A fully custom DeepWalk implementation (random walks + skip-gram with negative sampling) trained on a one-million–edge subset of Twitter-2010 to learn 256-dimensional embeddings for 400,000 nodes.
- A hybrid feature representation combining embedding operators (Hadamard, L_1, L_2^2, mean) with classical heuristics (CN, Jaccard, AA, RA).
- A supervised link prediction model using Random Forest that achieves an AUC of 0.8872, far surpassing a Common Neighbors baseline.
- Interactive functions for single-pair prediction and top-N recommendations, showcasing real-world applicability.

II. Background

A. Classical Link Prediction Heuristics

Given an undirected graph $G = (V, E)$ and two nodes $u, v \in V$, classical link prediction defines a similarity $s(u, v)$ based on neighborhoods $\Gamma(u)$ and $\Gamma(v)$:

$$\text{CN}(u, v) = |\Gamma(u) \cap \Gamma(v)|, \tag{1}$$

$$\text{Jaccard}(u, v) = \frac{|\Gamma(u) \cap \Gamma(v)|}{|\Gamma(u) \cup \Gamma(v)|}, \tag{2}$$

$$\text{AA}(u, v) = \sum_{w \in \Gamma(u) \cap \Gamma(v)} \frac{1}{\log |\Gamma(w)|}, \tag{3}$$

$$\text{RA}(u, v) = \sum_{w \in \Gamma(u) \cap \Gamma(v)} \frac{1}{|\Gamma(w)|}. \tag{4}$$

These measures are easy to compute and interpret, but rely solely on immediate neighborhoods, making them less effective when common neighbors are rare (as in sparse networks).

B. DeepWalk Embeddings

DeepWalk [1] learns node embeddings by performing random walks on G and treating the walks as sentences for a skip-gram model. For each node u and its context nodes v within a fixed window in the random walks, DeepWalk maximizes:

$$\sum_{u \in V} \sum_{v \in \mathcal{N}(u)} \log p(v \mid u), \tag{5}$$

where $\mathcal{N}(u)$ is the context set for u.

Using negative sampling, the conditional probability is optimized via:

$$\mathcal{L}_{u,v} = -\log \sigma(\mathbf{e}_u^\top \mathbf{e}_v) - \sum_{i=1}^{k} \mathbb{E}_{v_i \sim P_n(v)} \big[\log \sigma(-\mathbf{e}_u^\top \mathbf{e}_{v_i}) \big], \tag{6}$$

where $\mathbf{e}_u$ and $\mathbf{e}_v$ are embedding vectors, k is the number of negative samples, and P_n is a noise distribution.

III. Related Work

Liben-Nowell and Kleinberg [4] provide a foundational analysis of link prediction in social networks using heuristics based on node neighborhoods and shortest paths. While effective on small graphs, these methods do not scale well in accuracy to massive online networks.

DeepWalk [1] introduced random-walk-based embeddings, showing improvements in multi-label classification. Node2Vec [2] extends this framework with biased random walks that interpolate between breadth-first and depth-first exploration, capturing both homophily and structural equivalence. LINE [3] focuses on preserving first- and second-order proximity in large information networks.

Hybrid approaches that combine embeddings with additional structural or attribute-based features have been shown to improve performance in recommendation and link prediction tasks. Our work lies in this hybrid category: we jointly leverage learned embeddings and traditional heuristics.

IV. Dataset and Preprocessing

A. Twitter-2010 Graph

The Twitter-2010 dataset, released by the Stanford Network Analysis Project (SNAP), comprises:

- 41.7 million users (nodes),
- 1.47 billion directed follow edges.

An edge (u, v) in this directed graph means that user u follows user v.

To better understand the structure of our sampled subgraph, we analyze the degree distribution of the undirected graph. Figure 1 shows the empirical degree distribution on a log–log scale.

B. Subsampling Strategy

Training on the full graph is beyond the scope of a CPU-only environment. Instead, we uniformly sample:

- 1,000,709 edges,
- 1,245,256 unique nodes.

This provides a realistic but computationally manageable subgraph.

C. Undirected Conversion

To simplify link prediction and support undirected heuristics, we convert the directed subgraph into an undirected graph G_u where $\{u, v\} \in E_u$ if $(u, v) \in E$ or $(v, u) \in E$ in the original directed graph.

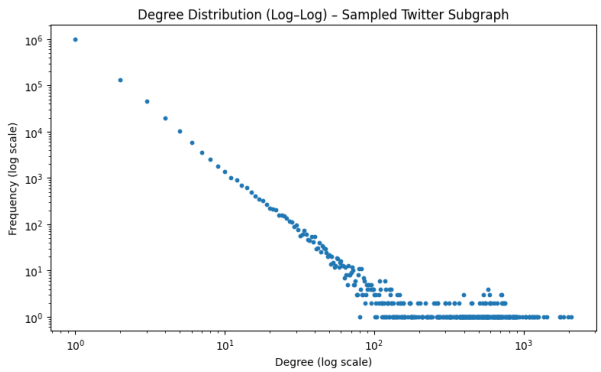

Fig. 1. Degree distribution of the sampled Twitter subgraph . The long tail indicates the presence of high-degree hub users, while most nodes have relatively small degree.

D. Positive and Negative Edge Sampling

We construct a labeled dataset as follows:

- **Positive edges**: randomly sampled from existing edges in G_u.
- **Negative edges**: node pairs (u, v) such that no edge exists between u and v in G_u.

We begin with 50,000 positives and 50,000 negatives and later filter based on the availability of embeddings.

V. Methodology

A. Pipeline Overview

The high-level pipeline consists of:

1) Generating random walks on G_u.
2) Training a DeepWalk skip-gram model to learn embeddings.
3) Constructing hybrid embedding + heuristic features for edge pairs.
4) Training a Random Forest classifier for link prediction.
5) Building interactive APIs for prediction and recommendation.

The overall architecture of the system is summarized in Figure 2.

B. Random Walk Generation

We generate truncated random walks starting from nodes in G_u. The main parameters are:

- Total walks: 1,000,000
- Maximum walk length: 20
- Window size: 5

Algorithm 1 outlines the process.

C. DeepWalk Skip-Gram Training

We treat each walk as a sentence and each node ID as a token. For a target node u and a context node v appearing within a window of size w in the walk, we form positive training pairs; negative samples are drawn from a noise distribution over nodes.

We implement a skip-gram model with:

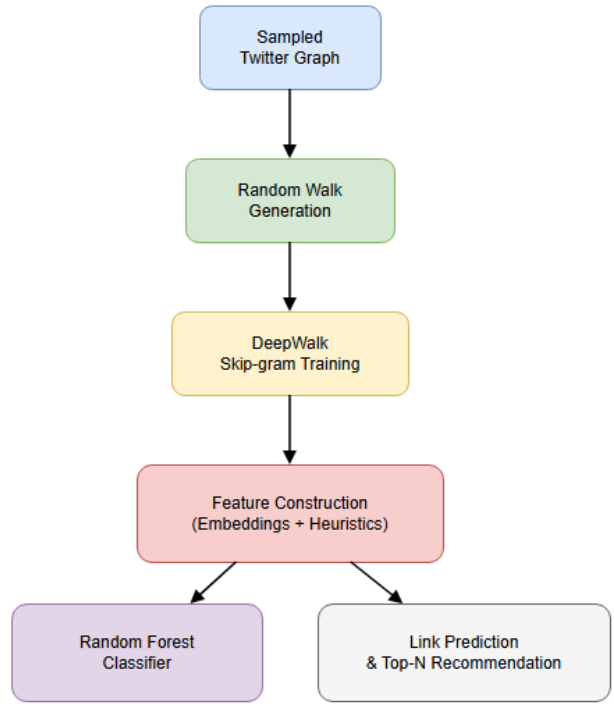

Fig. 2. Overall pipeline of the proposed link prediction system: from the sampled Twitter graph to random walks, DeepWalk training, feature construction, supervised classification, and downstream applications such as link prediction and top-N recommendations.

- Target embedding matrix $\mathbf{E}_t \in \mathbb{R}^{|V|\times 256}$,
- Context embedding matrix $\mathbf{E}_c \in \mathbb{R}^{|V|\times 256}$,
- Dot-product scoring, and
- BCEWithLogits loss with negative sampling.

We train for 6 epochs using the Adam optimizer with learning rate 10^{-3}.

D. Vocabulary Restriction

To control memory usage, we restrict the vocabulary to the 400,000 most frequently appearing nodes in the random walk corpus. This preserves the most structurally important nodes and avoids embeddings for extremely rare nodes that would rarely appear in training.

E. Hybrid Feature Construction

For a candidate edge (u, v) with embeddings $\mathbf{e}_u, \mathbf{e}_v \in \mathbb{R}^{256}$, we compute:

Embedding-based features:

$$\mathbf{h}_{\text{had}} = \mathbf{e}_u \odot \mathbf{e}_v, \tag{7}$$

$$\mathbf{h}_{L_1} = |\mathbf{e}_u - \mathbf{e}_v|, \tag{8}$$

$$\mathbf{h}_{L_2} = (\mathbf{e}_u - \mathbf{e}_v)^2, \tag{9}$$

$$\mathbf{h}_{\text{avg}} = \frac{\mathbf{e}_u + \mathbf{e}_v}{2}. \tag{10}$$

These contribute $4 \times 256 = 1024$ dimensions.

Heuristic features:

- Jaccard Coefficient,
- Adamic–Adar,

Algorithm 1 Random Walk Generation

Require: Graph $G = (V, E)$, number of walks R, walk length L

```
Initialize corpus C ← ∅
Let R_u be the number of walks per node (e.g., 10)
for each node u ∈ V do
    for i = 1 to R_u do
        w ← [u]
        current ← u
        for t = 1 to L − 1 do
            neighbors ← Γ(current)
            if neighbors is empty then
                break
            end if
            Sample v uniformly from neighbors
            Append v to walk w
            current ← v
        end for
        Append walk w to C
    end for
end for
return C
```

- Resource Allocation,
- Common Neighbors.

The final feature vector has dimension 1028.

F. Random Forest Classifier

We train a Random Forest classifier to predict whether an edge exists (label 1) or not (label 0). Key hyperparameters:

- Number of trees: 350,
- Max depth: 22,
- Minimum samples per leaf: 2,
- Criterion: Gini impurity.

Random Forest is chosen for its robustness to feature scaling, ability to model nonlinear decision boundaries, and interpretability.

G. Computational Complexity and Scalability

Random walk generation has complexity $O(R \cdot L)$, where R is the number of walks and L is the walk length. Skip-gram training with negative sampling is approximately linear in the number of generated pairs and in the embedding dimension. The memory footprint is dominated by the embedding matrices, which scale as $O(|V| \cdot d)$ where d is the embedding dimension. In our setting, the 400,000-node, 256-dimensional embedding matrix remains manageable on a modern laptop, but scaling to the full Twitter graph would benefit from GPU acceleration and additional optimization.

VI. Experimental Setup

A. Train–Test Split

After filtering edges to ensure both endpoints have embeddings, we obtain 11,547 labeled edges. We split them into:

- 70% training set (8,082 edges),
- 30% test set (3,465 edges),

with stratification to maintain a balanced class distribution.

B. Metrics

We evaluate ranking and classification performance using:

- Area Under ROC Curve (AUC),
- Precision,
- Recall,
- F1-score.

C. Implementation Details and Runtime

All experiments are implemented in Python using PyTorch for the skip-gram model, NetworkX for graph processing, and scikit-learn for the Random Forest classifier. Training is performed on a single laptop CPU without GPU acceleration. With the chosen configuration (approximately one million walks, 400,000-node vocabulary, 256-dimensional embeddings, and six epochs), the DeepWalk training phase requires on the order of a few hours of CPU time, while the Random Forest model trains in a few minutes.

D. Baselines

We compare against a Common Neighbors (CN) baseline, which scores each candidate edge by the count of shared neighbors in G_u.

VII. Results

A. Baseline Performance

Table I shows the performance of the CN baseline.

Metric	CN Value
AUC	0.50
Precision	0.50
Recall	1.00
F1-score	0.66

TABLE I
Common Neighbors baseline performance.

An AUC of 0.50 indicates that CN behaves like random guessing on this Twitter subset, highlighting the limitations of purely local heuristics in sparse networks.

To visualize this, Figure 3 shows the ROC curve of the Common Neighbors baseline.

B. DeepWalk + Heuristics + Random Forest

Our full hybrid model achieves the scores in Table II.

Metric	Value
AUC	0.8872
Precision	0.8707
Recall	0.7778
F1-score	0.8217

TABLE II
DeepWalk + Heuristics + Random Forest performance.

The improvement from AUC 0.50 to 0.8872 is substantial and demonstrates that embedding-based methods can capture predictive structural information that heuristics miss.

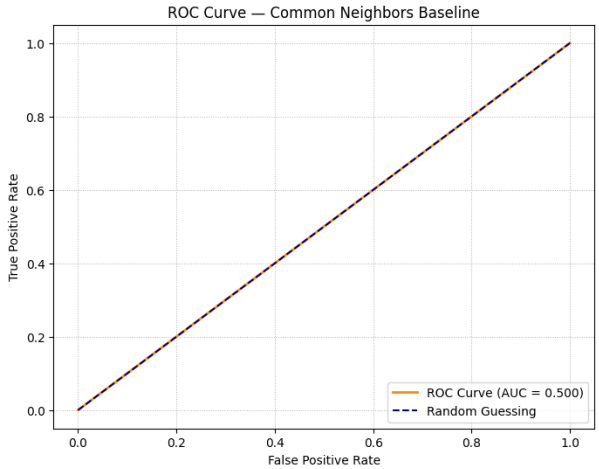

Fig. 3. ROC curve for the Common Neighbors (CN) baseline. The curve lies close to the diagonal, corresponding to an AUC of 0.50, which is equivalent to random guessing.

C. ROC Curve Analysis

Figure 4 shows the ROC curve for the proposed DeepWalk + Heuristics + Random Forest model on the test set.

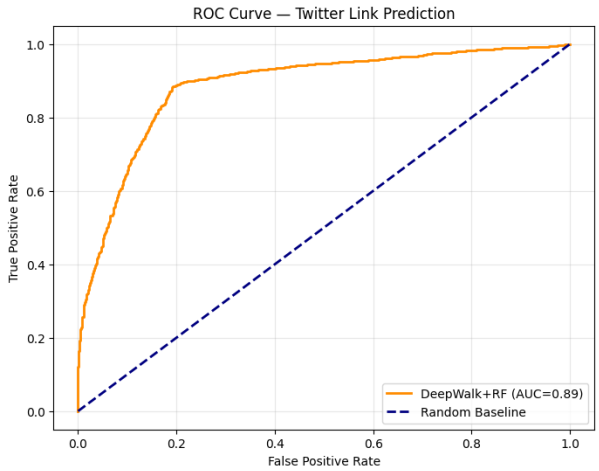

Fig. 4. ROC curve for the proposed DeepWalk + Heuristics + Random Forest model on the sampled Twitter subgraph. Compared to the baseline ROC curve in Figure 3, the hybrid model achieves a much higher true positive rate at nearly all false positive rates, with an AUC of 0.8872.

The ROC curve of the hybrid model lies significantly above the diagonal, with a steep slope at low false positive rates. By contrast, the CN ROC curve in Figure 3 is close to the diagonal, confirming its near-random behavior on this dataset.

D. Qualitative Case Studies

Using an interactive function `predict_link`(u, v), we inspect individual predictions:

- For high-probability edges (e.g., $p \approx 0.9$), we observe high cosine similarity between embeddings, multiple common neighbors, and strong AA/RA scores.
- For low-probability edges (e.g., $p \approx 0.2$), embeddings are nearly orthogonal, and heuristics are almost zero.

These patterns suggest that the model correctly combines local and global signals.

E. Top-N Recommendations

The `top_n_predictions`(u, N) function ranks candidate nodes based on model output probabilities. For several users, the top recommended accounts share strong structural similarity and overlapping neighborhoods, resembling "Who to Follow" suggestions. This qualitative behavior indicates that the learned model is suitable not only for offline evaluation but also for online recommendation tasks.

VIII. Discussion

A. Why the Hybrid Model Works

The hybrid approach is effective because:

- Embeddings capture long-range structural context, communities, and roles.
- Heuristics capture direct neighbor overlap and local proximity.
- Random Forest learns nonlinear interactions between these features.

Thus, even when heuristics alone fail, the embeddings provide discriminative power, and vice versa.

B. Limitations

The current work has several limitations:

- **Subgraph training**: We only use a sampled subgraph of Twitter; full-graph behavior may differ.
- **Static modeling**: We do not consider the temporal order of edge formation.
- **Cold-start nodes**: Nodes with very few interactions may still have less reliable embeddings.
- **CPU-only constraints**: GPU training could enable larger vocabularies and deeper models.

C. Ethical Considerations

While link prediction models can improve user experience through better recommendations, they may also amplify existing biases in the network. Highly connected or popular users are more likely to be recommended, potentially reducing visibility for new or marginalized users. Furthermore, the Twitter graph may contain bots, spam accounts, or abusive users, and our model does not explicitly distinguish these. In practical deployments, link prediction should therefore be combined with content-based and safety filters to avoid harmful or unsafe recommendations.

IX. Conclusion and Future Work

We presented a DeepWalk-enhanced, hybrid link prediction framework for large-scale social networks. By combining random-walk-based embeddings with classical graph heuristics and a Random Forest classifier, we achieved an AUC of 0.8872 on a one-million–edge subset of the Twitter-2010 graph, dramatically outperforming a Common Neighbors baseline.

Beyond numerical performance, we implemented interactive prediction and top-N recommendation tools that emulate real-world "Who to Follow" functionality. These demonstrate that our model is not only accurate but also practical for deployment.

Future work will explore:

- Biased random walks (Node2Vec) to better capture homophily and structural equivalence.
- Graph neural network architectures (GCN, GraphSAGE) for end-to-end learning.
- Temporal link prediction by modeling time-stamped edges.
- GPU-accelerated training on the full Twitter-2010 graph.

REFERENCES

[1] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online Learning of Social Representations," in *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2014.

[2] A. Grover and J. Leskovec, "node2vec: Scalable Feature Learning for Networks," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.

[3] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "LINE: Large-scale Information Network Embedding," in *Proceedings of the 24th International Conference on World Wide Web*, 2015.

[4] D. Liben-Nowell and J. Kleinberg, "The Link Prediction Problem for Social Networks," *Journal of the American Society for Information Science and Technology*, vol. 58, no. 7, pp. 1019–1031, 2007.